Programação Multithread com Pthreads em C

Guia de Referência — Passagem de Parâmetros, Retorno de Resultados e Condições de Corrida

---

0. Pré-requisitos de Compilação

Para utilizar a API Pthreads, é necessário incluir o cabeçalho `<pthread.h>`, além das bibliotecas padrão utilizadas nos exemplos. A compilação exige a flag `-lpthread` para vincular a biblioteca ao executável:

```
#include <pthread.h>   /* API Pthreads                    */
#include <stdlib.h>    /* malloc, free                    */
#include <stdio.h>     /* printf                          */

/* Compilação: */
/* gcc programa.c -o programa -lpthread                   */
```

1. Funções Principais da API

1.1 pthread_create — Criar uma nova thread

```
int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
```

Esta função cria uma nova thread dentro do processo atual. A nova thread começa sua execução invocando a função passada como argumento. Os parâmetros são:

- **`thread`**: ponteiro para uma variável `pthread_t`. Em caso de sucesso, a função armazena o identificador (ID) da nova thread neste endereço.
- **`attr`**: ponteiro para `pthread_attr_t` com atributos da thread (tamanho da pilha, política de escalonamento, etc.). Passa-se `NULL` para utilizar os atributos padrão.
- **`start_routine`**: ponteiro para a função que a thread executará. Essa função deve obrigatoriamente retornar `void *` e receber um único parâmetro `void *`.
- **`arg`**: argumento único passado à função da thread. Como é um `void *`, pode-se passar o endereço de qualquer tipo de dado e realizar o cast correto dentro da função. Se a função não precisar de argumentos, passe `NULL`.

Retorno: 0 em caso de sucesso; código de erro numérico em caso de falha (por exemplo, falta de recursos do sistema).

1.2 pthread_exit — Encerrar a thread atual

```
void pthread_exit(void *retval);
```

Esta função encerra a execução da thread que a chamou de forma limpa, sem encerrar o processo inteiro (como aconteceria com a chamada à função exit() da biblioteca padrão do C).

- **`retval`**: ponteiro (`void *`) para o valor de retorno da thread. Esse valor fica guardado pelo sistema operacional e pode ser recuperado por outra thread que chame `pthread_join`. Se não for necessário retornar nada, passe `NULL`.

Retorno: esta função não retorna ao chamador, pois a thread é encerrada imediatamente.

> **Nota:** *existe uma diferença importante entre as duas formas de encerrar a thread principal. Se a função `main` chamar `pthread_exit`, apenas ela é encerrada e o processo permanece vivo até que todas as threads filhas também terminem. Se, por outro lado, a `main` simplesmente* **retornar** *(ou chamar `exit()`), o processo inteiro é encerrado imediatamente, destruindo todas as threads filhas ainda em execução — independentemente de terem terminado ou não. Este é um dos principais motivos pelos quais o uso de `pthread_join` ou de `pthread_exit` na `main` é essencial.*

1.3 pthread_join — Aguardar o término de uma thread

```
int pthread_join(pthread_t thread, void **retval);
```

Esta função bloqueia a execução da thread chamadora até que a thread alvo termine (ou seja, até que ela chame `pthread_exit` ou retorne de sua função principal). É o equivalente ao `wait()` utilizado para processos.

- **`thread`**: identificador (ID) da thread a aguardar — o mesmo preenchido por `pthread_create`.
- **`retval`**: ponteiro para ponteiro (`void **`). Se não for `NULL`, a função copiará o valor de retorno da thread alvo para o local apontado. Passe `NULL` quando o valor de retorno não for necessário.

Retorno: 0 em caso de sucesso; código de erro em caso de falha.

> **Nota:** *realizar o join instrui o sistema operacional a liberar os recursos da thread (pilha, estruturas internas, etc.). Se uma thread não for submetida ao join — e não for criada como* ***detached*** *— seus recursos nunca serão liberados, caracterizando um vazamento de recursos conhecido como* **zombie thread**.

1.4 pthread_detach — Liberar recursos automaticamente

```
int pthread_detach(pthread_t thread);
```

Marca uma thread como *detached*. Uma thread nesse estado libera seus próprios recursos automaticamente ao terminar, sem necessidade de `pthread_join`. Em contrapartida, não é possível recuperar seu valor de retorno.

Há duas formas equivalentes de criar uma thread detached:

```
/* Forma 1 — via atributos, antes da criação: */
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
pthread_create(&tid, &attr, func, NULL);
pthread_attr_destroy(&attr);

/* Forma 2 — após a criação: */
pthread_create(&tid, NULL, func, NULL);
pthread_detach(tid);
```

Use `pthread_detach` quando a thread for independente e seu resultado não precisar ser coletado (por exemplo, threads de serviço ou de logging). Para qualquer thread cujo resultado ou término precise ser confirmado, prefira `pthread_join`.

2. Passagem de Parâmetros para Threads

Para compreender a programação multithread com a API Pthreads em C, é essencial entender que, embora as threads compartilhem o mesmo espaço de endereçamento, a comunicação de dados entre elas exige técnicas específicas para evitar erros de memória e inconsistências.

2.1 O Problema: Condição de Corrida (Race Condition)

O erro mais comum ao iniciar threads é passar o endereço de uma variável que muda de valor — como o contador de um laço `for`. Como todas as threads compartilham o mesmo espaço de endereçamento, elas enxergam o **mesmo endereço de memória**. Se o programa principal modifica essa variável antes de a thread lê-la, o resultado é imprevisível:

```
/* ERRO: todas as threads recebem &i                        */
/* ou seja, o mesmo endereço de memória da variável 'i'   */
for (int i = 0; i < 4; i++)
    pthread_create(&tid[i], NULL, func, &i);
```

Neste cenário, o laço continua incrementando `i` enquanto as threads são criadas. Algumas threads podem ser escalonadas somente depois que `i` já atingiu o valor final (4), fazendo com que todas leiam 4 em vez de receberem os valores 0, 1, 2 e 3 individualmente. O resultado depende do escalonador do sistema operacional e é **não determinístico** — caracterizando uma condição de corrida.

2.2 Condição de Corrida na Escrita — Variáveis Globais Compartilhadas

Além da leitura de parâmetros, um cenário igualmente perigoso — e muito frequente em produção — é o de múltiplas threads **escrevendo** simultaneamente em uma variável compartilhada sem sincronização. Considere o exemplo clássico de incremento concorrente:

```
int contador = 0;  /* variável global compartilhada         */

void *incrementar(void *arg) {
    contador++;    /* NÃO é atômica: leitura → soma → escrita */
    return NULL;   /* pode ser interrompida entre as 3 etapas */
}
```

A expressão `contador++` parece simples, mas é compilada em três operações distintas: leitura do valor, incremento e escrita. Se duas threads executarem essas etapas de forma intercalada, um dos incrementos pode ser **perdido silenciosamente**. A solução requer mecanismos de sincronização explícitos, como mutexes:

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;

void *incrementar(void *arg) {
    pthread_mutex_lock(&mtx);
    contador++;           /* região crítica protegida */
    pthread_mutex_unlock(&mtx);
    return NULL;
}
```

> **Nota:** *O uso de mutexes e outros mecanismos de sincronização (semáforos, variáveis de condição) está fora do escopo deste guia, mas é fundamental sempre que múltiplas threads acessam e modificam a mesma região de memória.*

2.3 Solução 1 — Cast de escalar para void *

Em vez de passar um *endereço* de memória, esta técnica embute o valor a ser passado diretamente nos bits do ponteiro. O valor de `i` é copiado para o ponteiro no momento da chamada a `pthread_create`, sem utilizar endereço algum:

```
/* Na main — passa o valor, não o endereço: */
for (long i = 0; i < 4; i++)
    pthread_create(&tid[i], NULL, func, (void *)(long)i);

/* Na função da thread — recupera o valor: */
void *func(void *arg) {
    long id = (long)arg;
    printf("Thread %ld\n", id);
    return NULL;
}
```

O cast intermediário para `long` em `(void *)(long)i` é obrigatório. Sem ele, converter um `int` diretamente para `void *` é tecnicamente inválido no padrão C (os tipos têm representações diferentes) e muitos compiladores emitem avisos ou erros. O cast para `long` garante que o inteiro seja primeiro estendido para o tamanho nativo de um ponteiro antes da conversão.

Limitação: esta técnica funciona apenas para valores que cabem em um ponteiro. Em arquiteturas de **32 bits**, sizeof(void *) == 4, limitando os valores inteiros seguros à faixa de um int de 32 bits. Em arquiteturas de 64 bits o espaço é mais que suficiente para qualquer int ou long.

2.4 Solução 2 — Struct com array de argumentos (Recomendado)

Para a passagem de múltiplos parâmetros com máxima segurança, utiliza-se uma estrutura de dados e um *array* alocado na pilha da `main`, garantindo que cada thread tenha seu próprio espaço fixo e independente na memória:

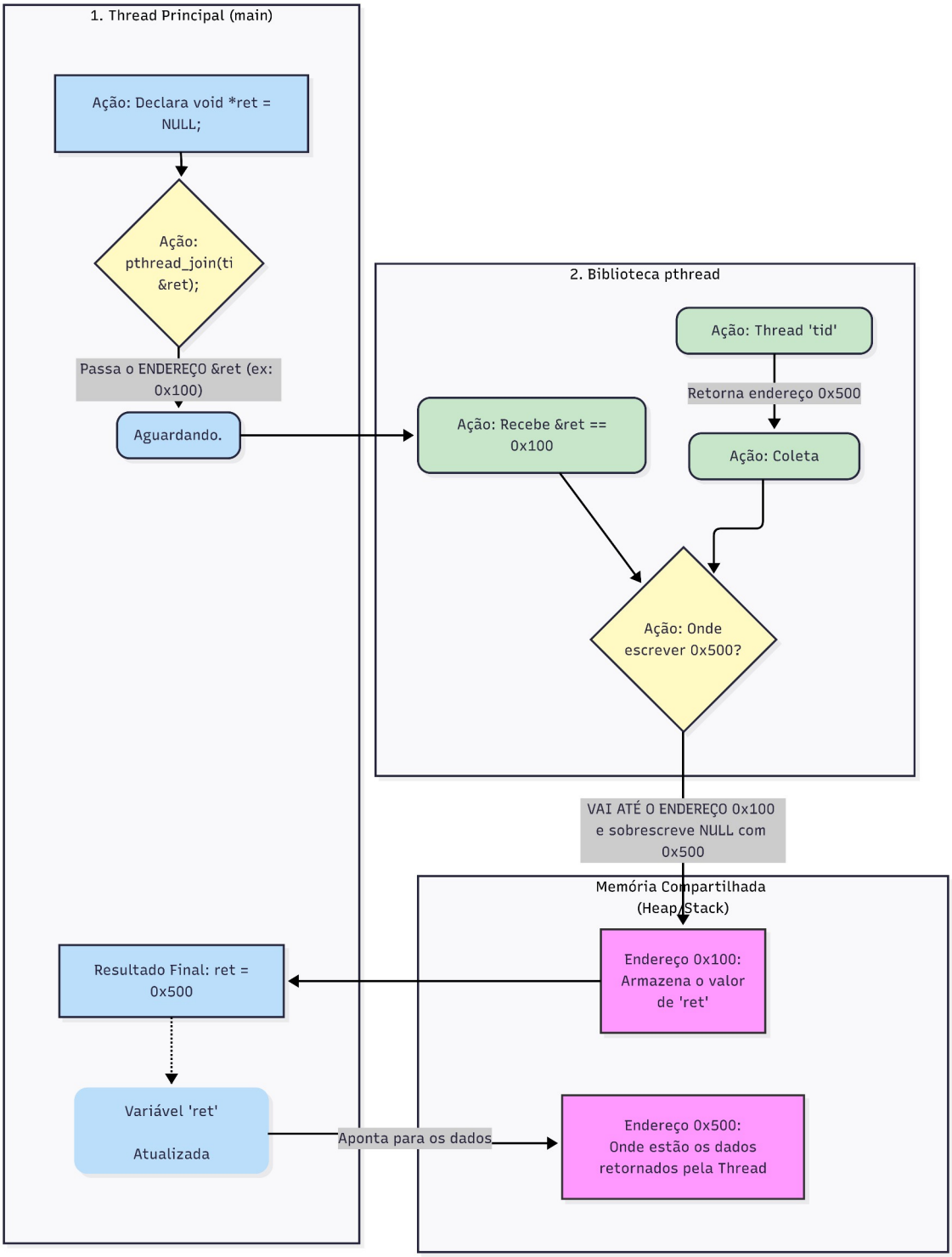
```
typedef struct {
    int id;
    int inicio;
    int fim;
} ThreadArgs;

void *worker(void *arg) {
    ThreadArgs *a = (ThreadArgs *)arg;
    printf("Thread %d: [%d, %d)\n", a->id, a->inicio, a->fim);
    return NULL;
}

int main(void) {
    pthread_t tid[4];
    ThreadArgs args[4];  /* array garante endereços independentes */

    for (int i = 0; i < 4; i++) {
        args[i].id     = i;
        args[i].inicio = i * 25;
        args[i].fim    = (i + 1) * 25;
        pthread_create(&tid[i], NULL, worker, &args[i]);
    }
    for (int i = 0; i < 4; i++)
        pthread_join(tid[i], NULL);
    return 0;
}
```

Esta abordagem é segura por três razões combinadas:

- Cada `args[i]` ocupa uma posição distinta na memória — os endereços são independentes entre si.
- A função `main` não modifica os elementos do array após criar as threads.
- O array persiste na pilha da `main` até o retorno da função, ou seja, pelo menos até o `pthread_join` ser concluído.

3. Retorno de resultados para o programa principal

Para capturar o resultado de uma thread, utiliza-se `pthread_join(thread, void **retval)`. O ponto crítico é **onde** o resultado é armazenado, pois o ciclo de vida da memória deve ser compatível com a sincronização entre *threads*. O parâmetro de retorno passado para a *thread* deve ser um endereço, pois a passagem é por referência, e não por valor. Mas como a *thread* vai retornar também um ponteiro, ou seja, um endereço, o parâmetro de retorno deve ser um ponteiro para ponteiro (void **).



3.1 O perigo da pilha — Nunca retorne variáveis locais

Nunca retorne o endereço de uma variável local da função da thread. Quando a thread encerra, seu "*stack frame*" é destruído e o ponteiro retornado torna-se inválido (*dangling pointer*):

```
void *func(void *arg) {
    int resultado = 42;
    return &resultado; /* ERRO: variável destruída ao encerrar */
}                      /* Comportamento indefinido (UB)!        */
```

3.2 A solução correta — Retorno via heap

A forma correta é alocar memória dinamicamente com `malloc`, pois a memória no heap persiste independentemente de qual thread a alocou. O ciclo de vida seguro é:

- A thread executa `malloc` e retorna o ponteiro via `return` ou `pthread_exit`.
- A `main` captura o ponteiro via `pthread_join`.
- A `main` deve obrigatoriamente executar `free` para evitar vazamento de memória (*memory leak*).

```
/* Função da thread — retorna resultado via heap */
void *calcular(void *arg) {
    int n = *(int *)arg;
    int *resultado = malloc(sizeof(int));  /* heap! */
    *resultado = n * n;
    return resultado;  /* ponteiro para heap é válido após a thread
encerrar */
}

int main(void) { /* main() */
    int n = 5;
    pthread_t tid;
    void *ret;   / Declara o ponteiro */

    pthread_create(&tid, NULL, calcular, &n);
    pthread_join(tid, &ret);     /* aguarda + passa endereço do ponteiro */
    printf("Resultado: %d\n", *(int *)ret); /* Acessa o valor retornado */
    free(ret);                        /* obrigatório: evita memory leak */
    return 0;
}
```

> **Nota:** *passar `NULL` como segundo argumento de `pthread_join` é válido e idiomático quando o valor de retorno da thread não é necessário: `pthread_join(tid, NULL);`*

3.3 Técnica Avançada — Struct Bidirecional (Entrada e Saída)

A solução mais elegante e escalável é usar uma única `struct` para entrada *e* saída. A struct é alocada na pilha da `main`, a thread escreve nos campos de saída e a `main` lê esses campos somente após o `pthread_join`, garantindo que a thread já terminou de escrever:

```
typedef struct {
    /* Entrada */
    int inicio;
    int fim;
    /* Saida */
    double soma;
    int status;   /* 0 = sucesso, != 0 = erro */
} ThreadData;

void *worker(void *arg) {
    ThreadData *d = (ThreadData *)arg;
    d->soma = 0;
    for (int i = d->inicio; i < d->fim; i++)
         d->soma += i;
    d->status = 0;
    return NULL;   /* resultado gravado diretamente na struct */
}

int main(void) {
    pthread_t  tid[4];
    ThreadData data[4];

    for (int i = 0; i < 4; i++) {
        data[i].inicio = i * 25;
        data[i].fim    = (i + 1) * 25;
        pthread_create(&tid[i], NULL, worker, &data[i]);
    }
    for (int i = 0; i < 4; i++) {
        pthread_join(tid[i], NULL);
        printf("Soma[%d]: %.0f (status: %d)\n",
               i, data[i].soma, data[i].status);
    }
    return 0;
}
```

Vantagens desta abordagem:

- Sem `malloc` extra — sem risco de memory leak.
- Entrada e saída encapsuladas em uma única estrutura — código limpo e legível.
- Sem condição de corrida no retorno: a `main` lê `data[i]` somente após o `pthread_join` garantir que a thread escreveu.
- Escalável: basta dimensionar o array `ThreadData data[N]` para qualquer número de threads.
- O campo `status` permite que a thread sinalize erros ou condições especiais ao chamador.

4. Resumo de Boas Práticas

Faça sempre:

- Chame `pthread_join` para cada thread criada. É essencial para evitar *zombie threads* e garantir que os resultados estejam prontos para leitura.
- Use arrays de structs ou `malloc` para garantir que cada thread tenha seu próprio espaço de memória — nunca passe o endereço de uma variável de laço diretamente.
- Acesse os campos de resultado somente após confirmar o término da thread com `pthread_join`.
- Chame `free` na thread chamadora após capturar o retorno, se a thread utilizou `malloc` para alocar o resultado.
- Considere `pthread_detach` para threads independentes que não precisam retornar resultados, evitando a necessidade de `pthread_join`.

Nunca faça:

- Nunca retorne o endereço de uma variável local da função da thread — o stack frame é destruído ao encerrar, gerando **comportamento indefinido** (*undefined behavior*).
- Nunca compartilhe o endereço de uma variável modificada pela thread criadora sem sincronização — use mutexes ou outra primitiva de sincronização para proteger a região crítica.
- Nunca assuma que uma thread executará antes de outra. Se houver dependência de ordem, use mecanismos explícitos (mutexes, semáforos, variáveis de condição).
- Nunca converta `int` diretamente para `void *` sem o cast intermediário para `long` — use sempre `(void *)(long)i`.

5. Tabela Comparativa de Estratégias

Estratégia	Race Cond.?	Múlt. Args	Retorno	Quando usar
`&variável de laço (i)`	SIM ✗	Não	—	NUNCA — apenas didático
`Cast (void*) (long)i`	Não ✓	Não	Não diretamente	1 arg inteiro ou enum simples
`Array de structs — &args[i]`	Não ✓	Sim	Campo no struct	Múltiplos args + retorno
`malloc() por thread`	Não ✓	Sim	Heap: return ptr	Threads independentes; free obrigatório
`Struct bidirecional`	Não ✓	Sim	Campos no struct	Solução mais completa e escalável

✦ **Recomendação geral:** prefira a struct bidirecional para máxima clareza e segurança. Reserve malloc() para threads genuinamente independentes onde o array de structs não é conveniente.